- From the previous scenario exercise, you provided with the AWS Cost Calculator and Auto-scaling, you have been asked by the organization to explain your recommendations for DR & Resiliency. You have been tasked to identify the following:
  - What is your recommended Recovery Time Objective (RTO) and Recovery Point Objective (RPO)?
  - What would be necessary for your project to ensure a solid DR plan?
  - What would you do to ensure your application is resilient?
  - Would you use certain managed services, deployment strategies or other precautions to avoid an outage?
  - What type of "chaos engineering" techniques would you use to test resiliency?
  - You have 15-20 minutes to discuss your recommended plan
- Download the template from Assignments > Activity - DR & Resiliency Recommendations
  - 1 submission per team
- The scenarios can be found in Assignment > Activity - Cost Estimating Scenarios

RECAP

- Reality of Cloud Systems
  - Cloud systems are not immune to failure—major outages (AWS, Azure, Google) highlight risks
  - Even small errors (e.g., misconfigurations, bad deployments) can cause widespread disruption
  - Downtime is costly (>$300K/hour) and impacts revenue, reputation, and customer trust
- Key Concepts
  - **RTO** (Recovery Time Objective): Maximum acceptable downtime
  - **RPO** (Recovery Point Objective): Maximum acceptable data loss
  - High availability targets (e.g., 99.999%) require strong design and investment
- Core Strategy
  - Downtime is inevitable—focus on prevention and rapid recovery
  - Plan for:
    - Disaster recovery
    - Resilient architecture
    - Controlled deployment strategies
    - Failure testing

Cloud Observability and Monitoring

Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology

Why Observability Matters

- *"What happens after you deploy your application successfully to the cloud?"*
- Deployment is only the beginning as real success depends on reliability, performance, and user experience over time
- Without observability, teams lack the context to understand why systems fail, making root cause analysis slow and reactive
- Comprehensive data from logs, metrics, and traces provides actionable insight into system behavior, enabling faster troubleshooting and informed decisions
- Proactive detection of anomalies and dependencies helps prevent cascading failures and ensures resilience at scale

Dec 2021 – AWS (from earlier this semester)

Amazon Says 'Unexpected Behavior' Caused Huge Cloud Outage

- Company's technical statement pledges to learn from event
- Expert says 'bad piece of code' sparked a 'snowball effect'

Amazon.com Inc. said automated processes in its cloud computing business caused cascading outages across the internet this week, affecting everything from Disney amusement parks and Netflix videos to robot vacuums and Adele ticket sales.

"Basically, a bad piece of code was executed automatically, and it caused a snowball effect," Forrester analyst Brent Ellis said. The outage persisted "because their internal controls and monitoring systems were taken offline by the storm of traffic caused by the original problem."

Source: https://www.bloomberg.com/news/articles/2021-12-10/amazon-says-massive-cloud-outage-caused-by-unexpected-behavior

Monitoring vs. Observability

- Monitoring detects known failure conditions using predefined thresholds
  - It is commonly tied to basic availability or error-rate metrics
  - Answers questions like:
    - Is CPU too high?
    - Is something broken?
- Observability enables understanding of why failures or degradations occur
  - Essential for diagnosing complex, distributed failures
  - Answers questions like:
    - Which users are impacted?
    - What changed?
    - Where is the bottleneck?

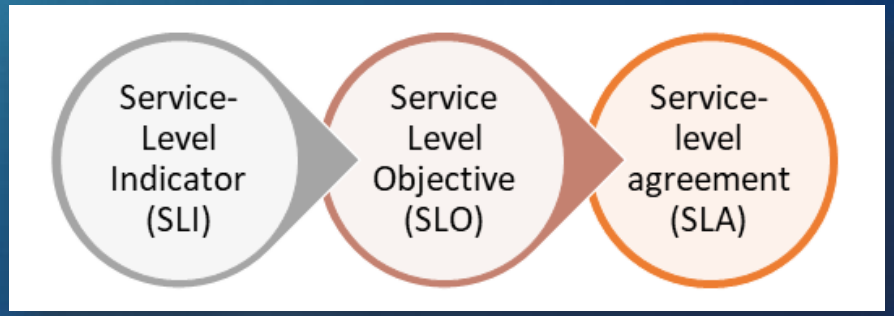
Source: https://icinga.com/blog/observability-vs-monitoring-enhancing-not-replacing/

Monitoring and Observability Working Together

- Monitoring acts as the early warning system, alerting you when something goes wrong
- Observability serves as the diagnostic toolkit, helping you understand why it went wrong
- Most real-world incidents begin with a monitoring alert and end with observability-driven root cause analysis (RCA)
- Most incidents follow this flow:
  - Monitoring alarm fires – Detects symptoms and signals that something is wrong
  - Observability tools investigate – Provide deep insights into system behavior to uncover the root cause
  - Insights feed back – Improve monitoring thresholds and refine system design for greater resilience

Introducing SLI, SLO and SLA

- Service Level Indicator (SLI)
  - A measurable signal that reflects user experience (e.g., latency, error rate, availability)
- Service Level Objective (SLO)
  - A target or threshold for an SLI (e.g., 99.9% of requests complete in under 500 ms)
- Service Level Agreement (SLA)
  - A formal promise to customers, often with penalties if unmet
- Observability/Monitoring Connection
  - Monitoring tracks SLIs in real time
  - Observability explains *why* SLIs are trending toward or away from SLOs
  - SLAs depend on both being effective



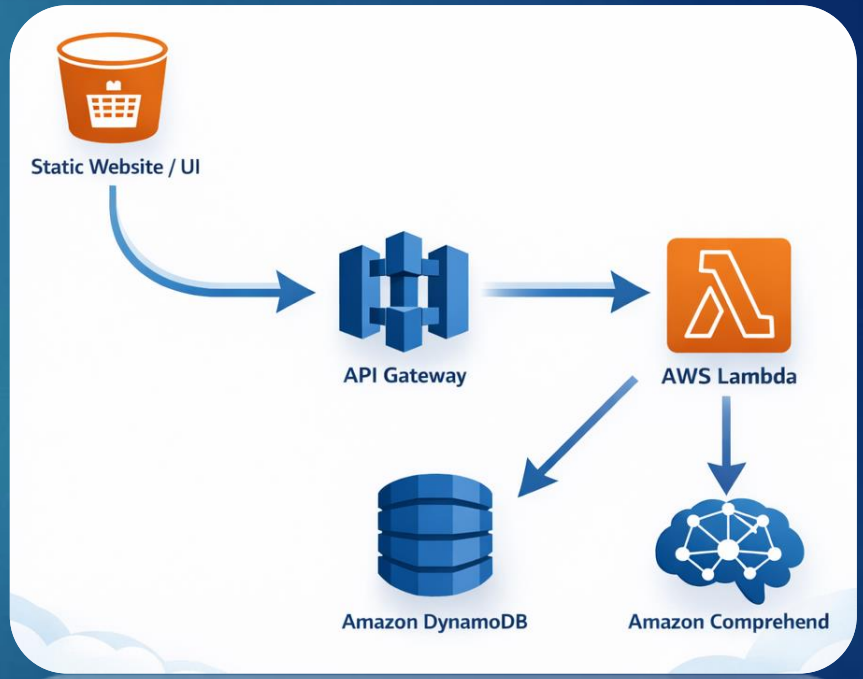
Source: https://www.linkedin.com/pulse/sre-concepts-part-2-slislo-marcel-koert/

Why Cloud Systems Require Observability

- Cloud applications are distributed, dynamic, and failure-prone by design
- Failures are often partial and affect only subsets of users
- SLIs can degrade gradually (latency creep, retries, throttling) before outages occur
- Without observability:
  - Teams miss early warning signs that SLOs are at risk
  - Troubleshooting becomes slow, reactive, and guess-driven
  - SLA breaches lead to costly consequences and damaged trust

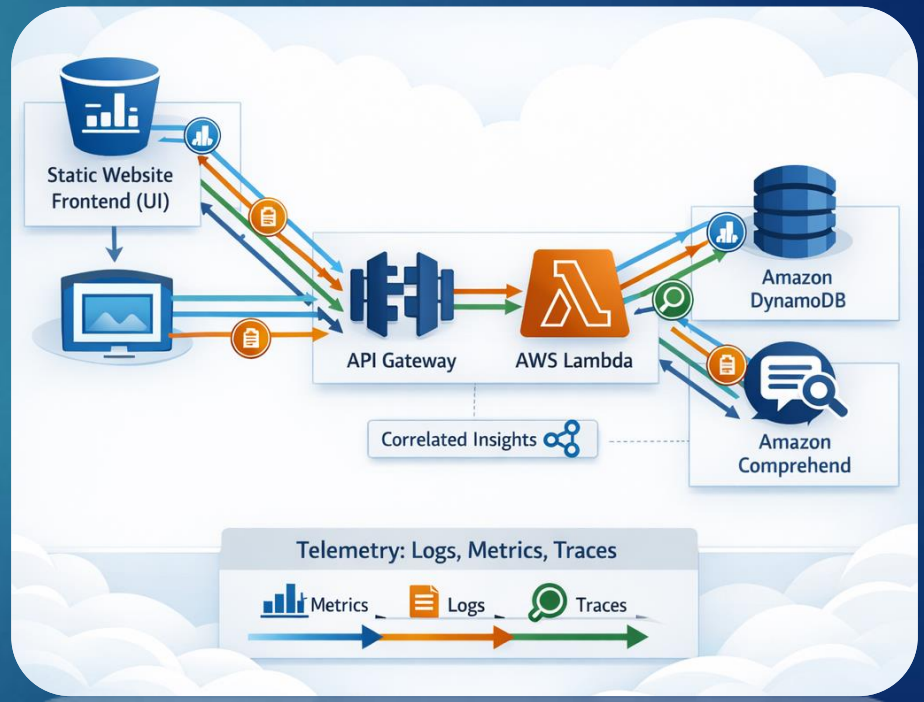
Example: Monitoring Only

- You built a serverless application built with: Amazon API Gateway, AWS Lambda, DynamoDB and Amazon Comprehend
- Defined SLO:
  - 99% of requests should complete within 500 ms
- Monitoring only:
  - Alarm fires when latency crosses a threshold
  - Dashboard shows elevated response times
- What's missing:
  - Which dependency caused the slowdown?
  - Whether all users or only some users are affected?
  - What changed recently?
- Teams know there's a problem, but diagnosis is slow and guess-driven



Example: Monitoring + Observability

- With observability added:
  - Monitoring detects the SLO risk early
  - Metrics show which services are degrading
  - Logs reveal errors or retries
  - Traces identify where latency is introduced
- Result:
  - Faster root-cause analysis
  - More confident fixes
  - Reduced mean time to recovery (MTTR)



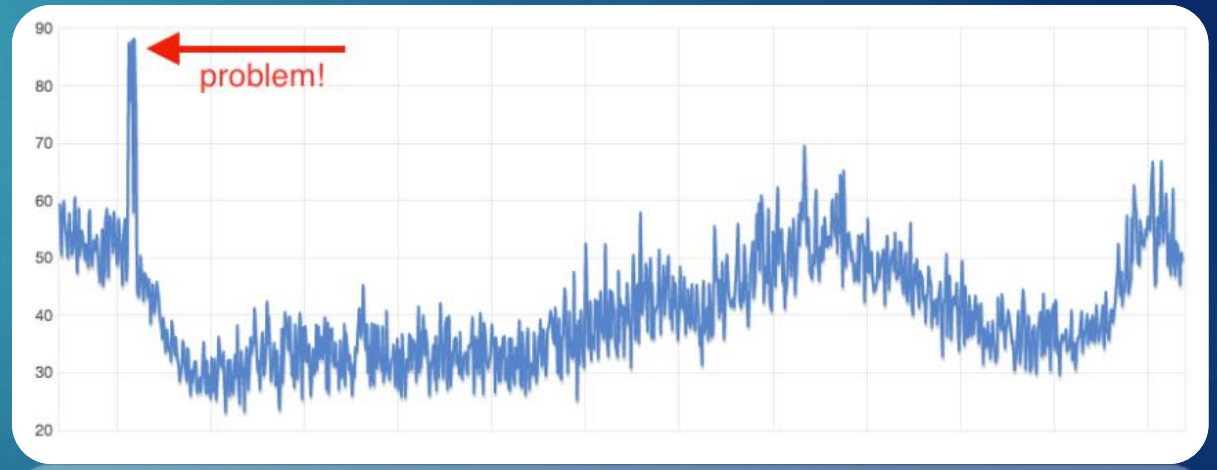
The 3 Pillars of Observability

- Metrics
  - The foundation for measuring system health
  - Primary source for Service Level Indicators (SLIs)
  - Used to assess compliance with Service Level Objectives (SLOs)
- Logs
  - Provide detailed context once an alert fires
- Traces
  - Visualize the complete journey of a request
  - Reveal dependencies and bottlenecks across distributed services

- **Monitoring** focuses on tracking metrics, while **observability** provides the context needed to understand their root causes

Monitoring and Metrics in the Cloud – CloudWatch

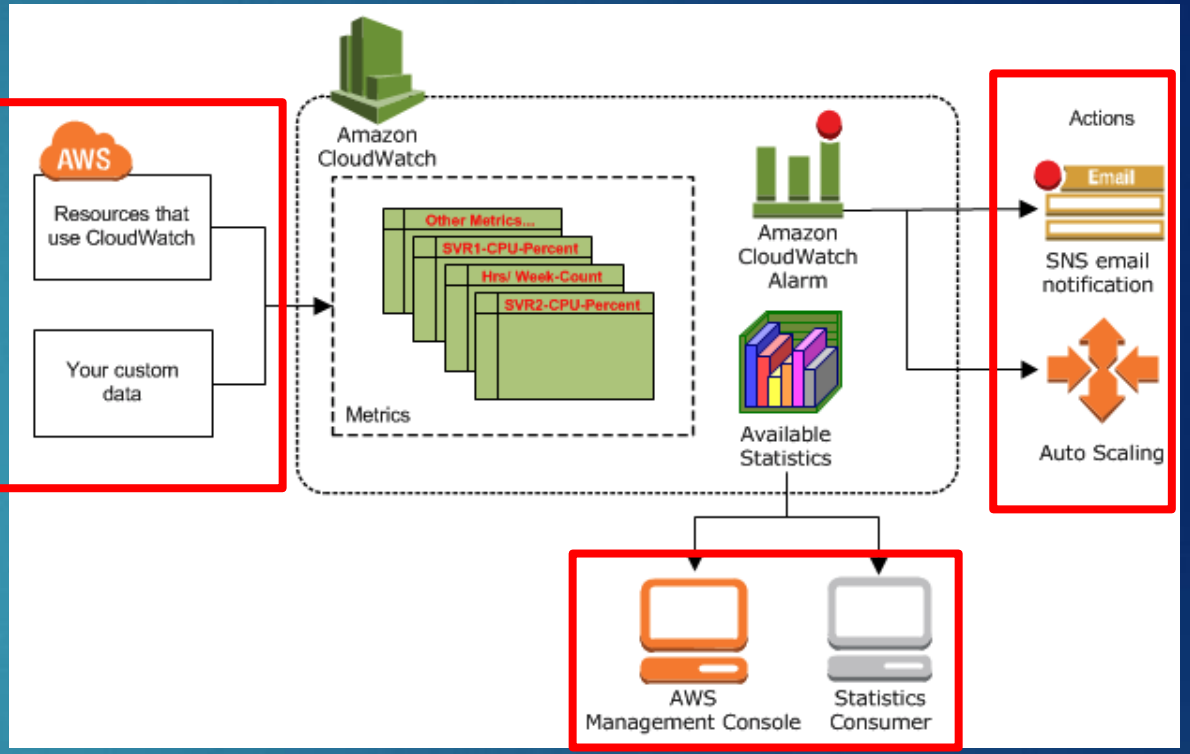
- CloudWatch is a service for monitoring AWS resources and the applications you run on AWS
- It allows developers, system architects, and administrators to monitor their AWS applications in the cloud, in near-real-time
- It is automatically configured to provide metrics on request counts, latency, and CPU usage
- You can use CloudWatch to collect and track metrics, collect and monitor log files, set alarms and automatically react to changes in AWS resources



- Amazon CloudWatch is primarily a monitoring service, but it also supports observability

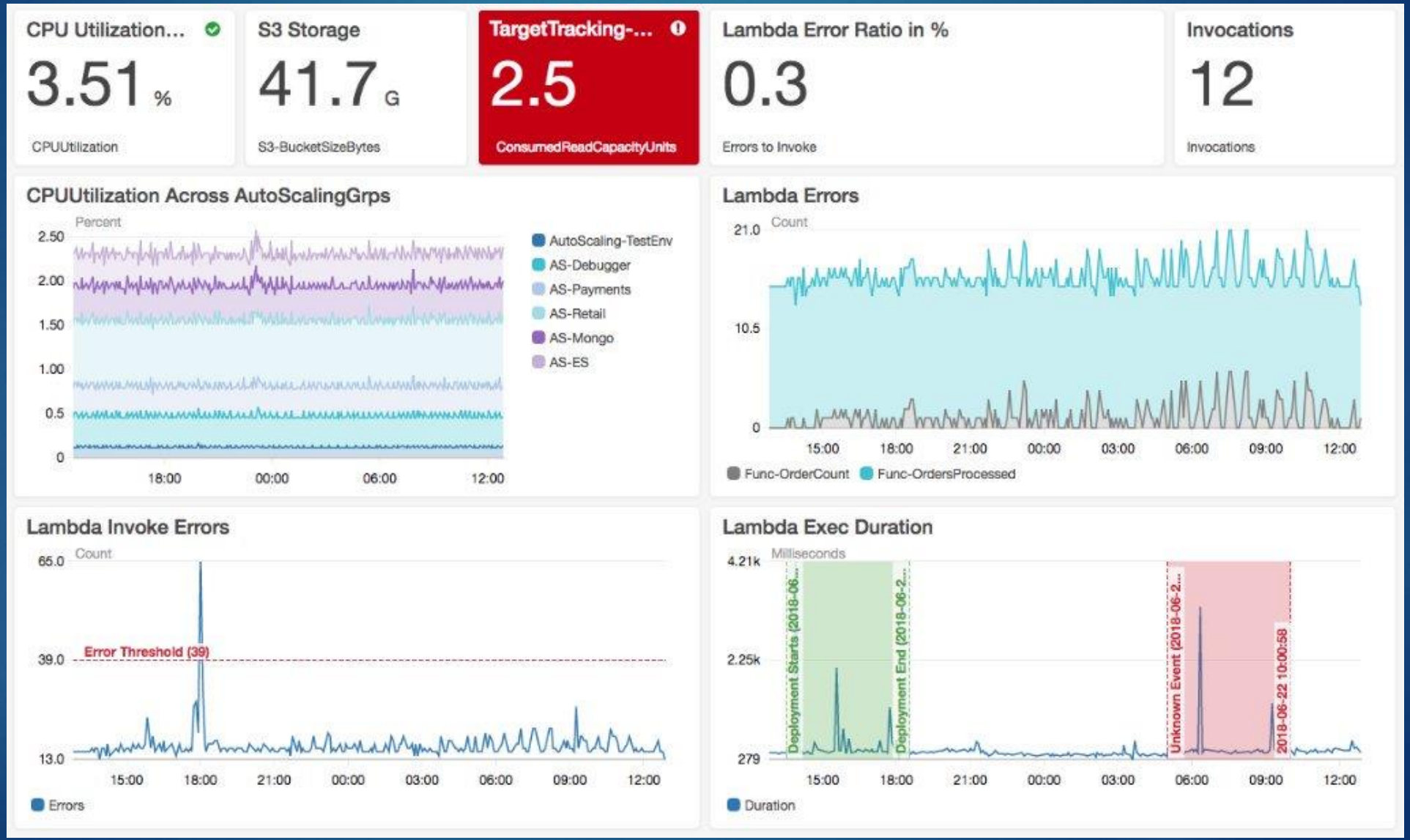
CloudWatch – How it works

- CloudWatch is central metrics repository
- AWS services (like EC2) publish metrics to CloudWatch, and you can retrieve statistics from these metrics
- Metrics can be used to calculate statistics and visualize data in the CloudWatch console (e.g., Dashboards)



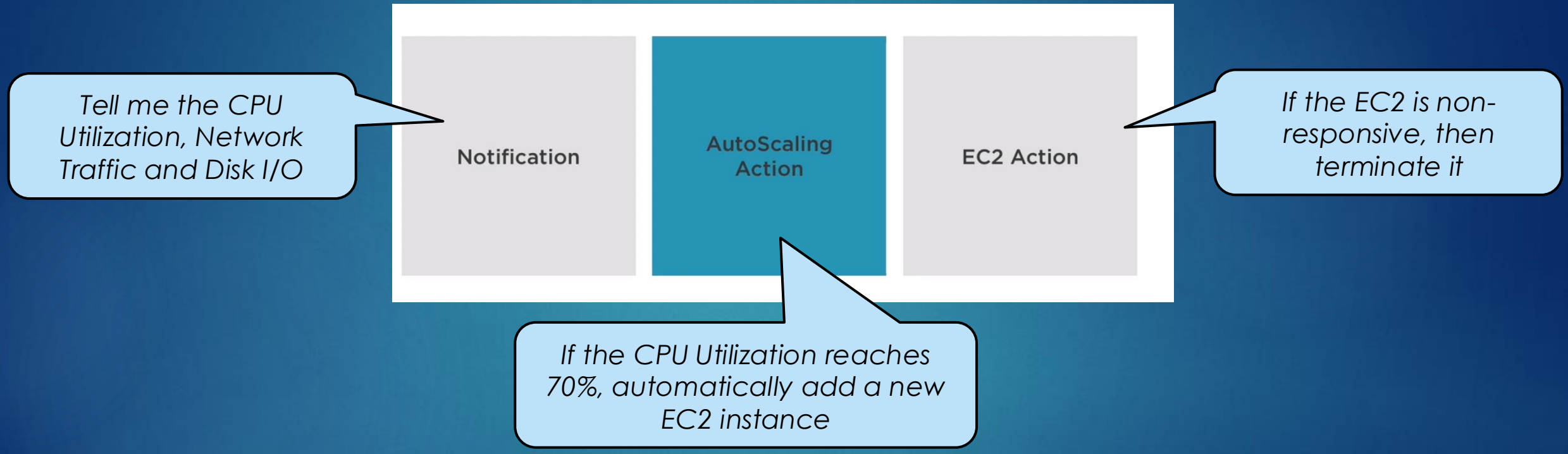
- You can set up alarms to automatically stop, start, or terminate EC2 instances when specific conditions are met
- Alarms can also trigger Auto-Scaling actions or send notifications via Amazon SNS

CloudWatch – Dashboard



CloudWatch – Alarm Actions Example

- Assume we have set up an alarm for an EC2 Instance…



- Actions can be combined to provide automated resiliency and scalability capabilities
- Example: *"If the EC2 instance reports a status of StatusCheckFailed_System, reboot the EC2 and send a notification to the support team"*

Advantages of CloudWatch

- Unified Dashboard
  - View all collected data from distributed applications in a single CloudWatch dashboard
- Cost Optimization
  - Set alarms and automate actions when thresholds are breached, reducing unnecessary AWS costs
- Detailed Insights
  - Gain visibility into AWS services and applications, including metrics like memory, CPU, and capacity utilization
- Performance Optimization
  - Use logs and metrics to fine-tune applications and resources for maximum efficiency and throughput

CloudWatch Limitations

- **Short Data Retention** - Metrics are stored for only 2 weeks
- **AWS-Only Monitoring** - Cannot track non-AWS services or user-managed infrastructure
- **Limited Visualization** - Graphs and widgets offer basic functionality.
- **Low Frequency in Basic Monitoring** - Data collected every 5 minutes, which may be insufficient for critical environments
- **Higher Cost for Detailed Monitoring** - Increased granularity comes with additional charges
- **Scalability Concerns** - Large organizations often pair CloudWatch with industry-standard monitoring tools for broader coverage

Cloud Monitoring Solutions

- CloudWatch provides core monitoring for application health and performance within AWS
- For broader visibility, many organizations integrate third-party tools (e.g., Datadog, New Relic, Prometheus) that:
  - Combine CloudWatch data with other sources
  - Deliver holistic reporting across multi-cloud and on-prem environments
  - Offer advanced analytics, visualization, and alerting capabilities

Source: https://sprinto.com/blog/best-cloud-monitoring-tools/

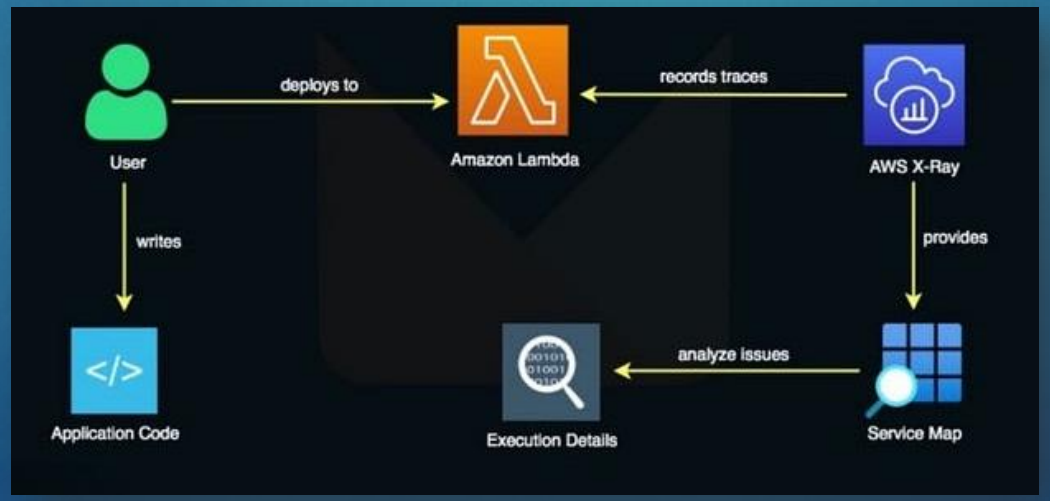
Datadog + CloudWatch – Dashboard

AWS Observability – Beyond Monitoring

- Builds on CloudWatch metrics and alarms
- Shifts the focus from "Is it broken?" to "Why did it happen?"
- Core Components:
  - CloudWatch Logs – Centralized application and service logs for context
  - AWS X-Ray – Distributed tracing for end-to-end visibility
  - Correlation – Connects metrics, logs, and traces to uncover root causes
- Key Point:
  - Observability transforms alerts into actionable understanding

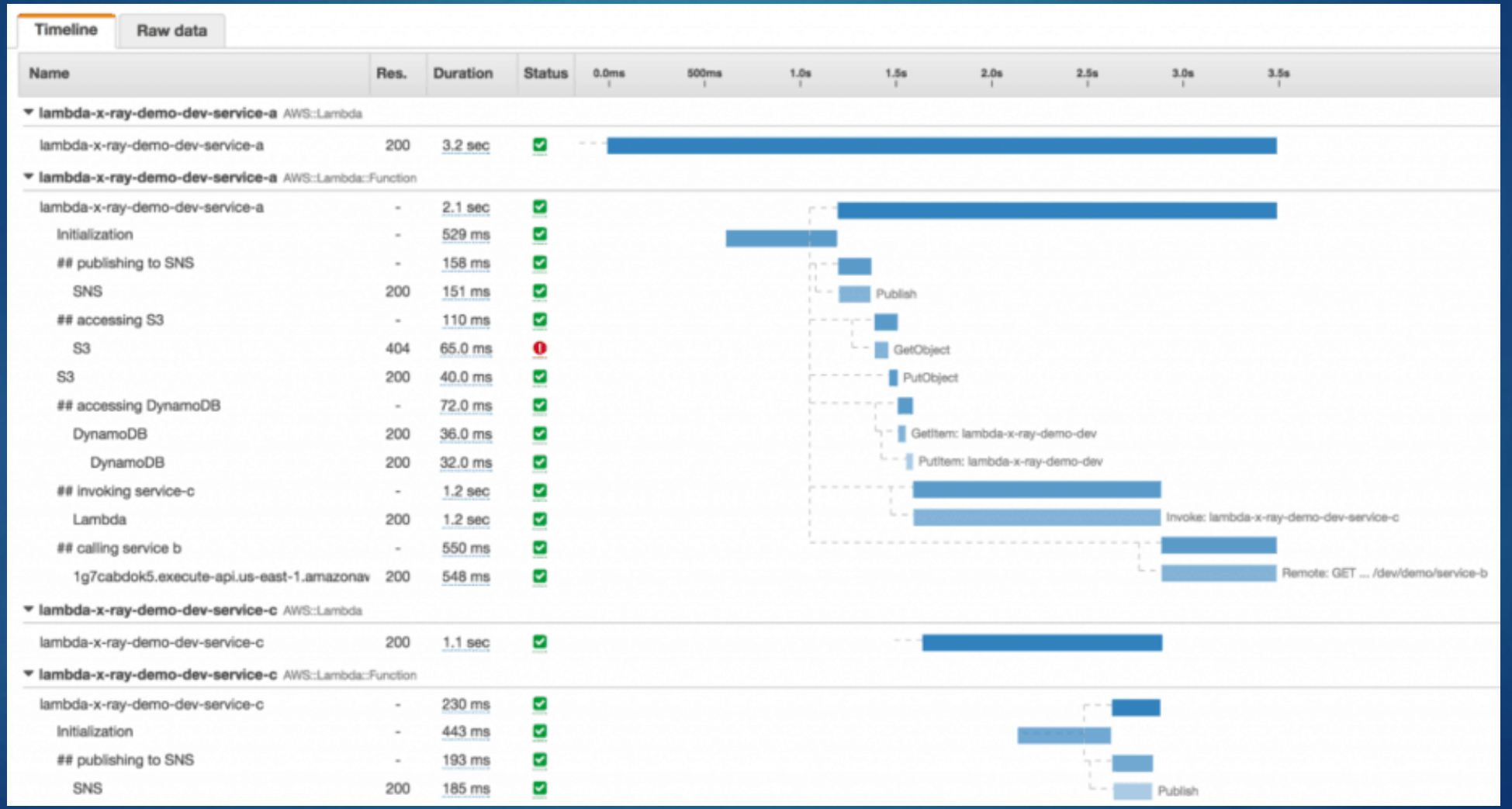
AWS X-Ray – Distributed Tracing on AWS

- X-Ray traces units of work as they flow through distributed applications, not limited to HTTP API calls
- Traces units of work across API Gateway, Lambda. EC2, DynamoDB and events from SQS, SNS, EventBridge, or S3
- X-Ray captures latency, errors, and downstream dependencies across all services involved in a request
- Metrics show that something is slow, X-Ray shows where and why



Source: https://aws.plainenglish.io/hands-on-implementing-aws-lambda-with-aws-x-ray-c284029cb470

AWS X-Ray – Screenshot



OpenTelemetry – The Observability Standard

- OpenTelemetry (OTel) is a vendor-neutral framework for collecting key observability signals:
  - Metrics – Performance and health indicators
  - Logs – Detailed event and error context
  - Traces – End-to-end request flow across services
- Defines how telemetry data is generated, structured, and propagated
- Eliminates vendor lock-in and ensures portability across platforms
- OpenTelemetry standardizes observability signals and platforms like AWS consume them

OpenTelemetry + AWS End-to-End Flow

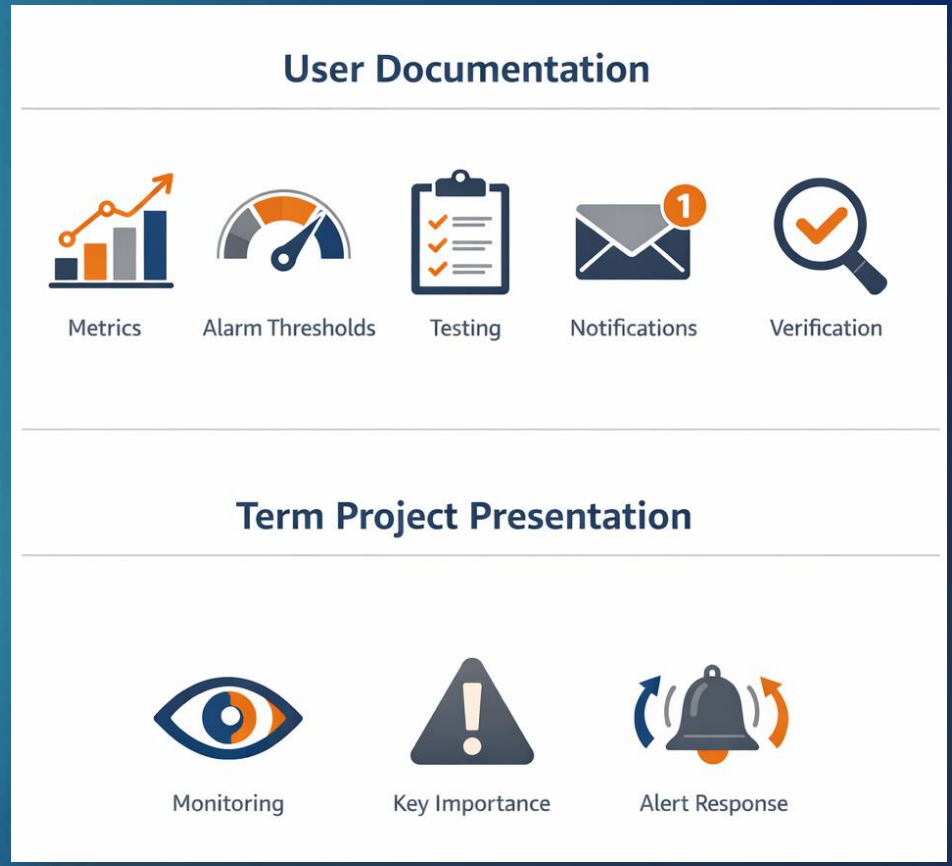
- Applications emit telemetry using OpenTelemetry SDKs
- AWS services consume this data:
  - Metrics → CloudWatch for monitoring and SLO compliance
  - Traces → AWS X-Ray for distributed request analysis
- Operational Flow:
  - OpenTelemetry instruments application code
  - Metrics feed CloudWatch alarms (monitoring)
  - Alarms signal SLO risk
  - Logs and X-Ray traces explain root cause
  - Insights improve future monitoring and system design
- This enables consistent observability across AWS and non-AWS systems

Observability Requirements for the Term Project

- Teams must use Amazon CloudWatch to monitor application health, performance, and failures
- Logging
  - Send meaningful logs (requests, errors, downstream calls) from Lambda, Fargate, EC2 to CloudWatch Logs
- Metrics
  - Define at least one custom application metric (e.g., error rate, latency). No dashboard required
- Alerting
  - Create one CloudWatch alarm linked to a metric and send notifications via Amazon SNS (e.g., email)

Observability Requirements for the Term Project

- In your User's document, teams must explain:
  - What metric is monitored
  - Alarm threshold
  - How to trigger the alarm in testing
  - How notifications are delivered
  - Instructors must be able to verify alerts during grading
- For the Term Project presentation, teams should be prepared to explain:
  - What you monitor
  - Why it matters
  - How alerts help detect/respond to issues in production



CloudWatch Activity

- You will create a simple alarm and dashboard to monitor an application we've ran before (WordPress)
- The dashboard will monitor a basic metric like CPU usage
- You will be doing a simple "chaos engineering" task of overwhelming the CPU to trigger an alarm
- Located at: Assignments > Activity– Create a CloudWatch Dashboard and Alarm